SRD: arquitectura multi-tenant para la gestión digital de Juntas de Acción Comunal

Juan Sebastián González Horta
Servicio Nacional de Aprendizaje (SENA) · Neiva, Colombia

Resumen— Las Juntas de Acción Comunal necesitan herramientas que organicen registros, actividades, recursos y comunicaciones sin mezclar información entre organizaciones. Este estudio de caso tecnológico, de carácter aplicado y descriptivo, examina SRD hasta el 19 de octubre de 2026 mediante revisión de código, migraciones, pruebas de servicios, registros operativos y recorridos de navegador con datos sintéticos. El prototipo cuenta con un Gateway, diez microservicios Laravel, bases MySQL por dominio y una interfaz web React; Android y Windows son clientes parciales. Sobre una misma imagen web servida por Docker aprobaron 33 archivos distintos de pruebas de navegador de 140 disponibles; los 107 restantes carecen de ejecución final registrada en esa imagen. Esta cifra no representa una tasa de aprobación de toda la batería ni el avance porcentual del sistema. Persisten bloqueos de carga por firmas antivirus vencidas y pendientes de regresión completa, autorizaciones de escritura por rol, usuarios reales y TLS/TURN de producción. La investigación expone progreso técnico verificable y límites de aceptación.

Palabras clave— multi-tenancy, Juntas de Acción Comunal, microservicios, auditoría, seguridad, investigación aplicada

I. INTRODUCCIÓN

La acción comunal colombiana tiene una estructura organizativa propia, reconocida en la Ley 2166 de 2021 [7]. En el trabajo administrativo cotidiano, una junta necesita conservar afiliaciones, registrar actividades, ubicar documentos, consultar sus recursos y saber quién modificó un dato. Cuando esas operaciones dependen de archivos y aplicaciones separados, se dificulta comprobar versiones, permisos y continuidad del registro. SRD parte de ese problema operativo y lo aborda como un proyecto de desarrollo de software orientado a servicios, no como reemplazo del régimen jurídico de las organizaciones.

La pregunta que orientó el incremento es la siguiente: ¿cómo construir una plataforma digital compartida para varias Juntas de Acción Comunal que permita operar por módulos y preserve la separación de datos entre organizaciones con una trazabilidad comprobable? La hipótesis de diseño es que un Gateway que verifique la sesión, una identidad de junta obligatoria en cada dominio y la publicación diferida de eventos permiten implementar progresivamente esas condiciones.

El objetivo general fue estructurar y validar técnicamente un incremento funcional de SRD para la administración de varias juntas. Los objetivos específicos fueron delimitar el problema y sus reglas; diseñar las fronteras entre clientes, servicios y datos; implementar recorridos representativos en web y clientes parciales; y documentar pruebas y fallas que permitan distinguir avance de aceptación definitiva.

II. MARCO DE REFERENCIA Y VIGILANCIA TECNOLÓGICA

La arquitectura multi-tenant permite que múltiples organizaciones utilicen una plataforma compartida, pero exige decisiones explícitas sobre partición de información, personalización y recuperación. Tsai y colaboradores [1] identifican estos problemas para SaaS; Krebs, Momm y Kounev [2] diferencian la tenencia del simple uso compartido de recursos. Por ello, que existan varias bases de datos en SRD no significa que haya una base por junta: las bases separan dominios; el atributo de organización separa las filas y operaciones dentro de cada dominio.

Los riesgos de seguridad no se eliminan por centralizar el control de acceso. Pervez y colaboradores [3] estudian la distribución y la seguridad en SaaS multi-tenant, y Singh y colaboradores [4] proponen un modelo de protección entre arrendatarios. Krebs y Loesch [5] advierten que el aislamiento de rendimiento es distinto del aislamiento de datos. Reinhardt y colaboradores [6] tratan decisiones de aprovisionamiento mixto. Estas publicaciones justifican analizar seguridad, responsabilidad de datos y comportamiento bajo carga como problemas separados.

La vigilancia tecnológica se complementó con referentes oficiales: la Ley 2166 [7] delimita el contexto comunal; OWASP ASVS 5.0 [8] ofrece requisitos de verificación de aplicaciones web; y NIST SP 800-63B-4 [9] orienta criterios de autenticación. No se afirma certificación de conformidad: estas referencias sirven como base para seleccionar controles verificables, entre ellos autorización en servidor, sesión y verificación adicional.

En soluciones existentes, CiviCRM [10] demuestra el uso de herramientas abiertas para entidades sin ánimo de lucro; Odoo [11] y ERPNext [12] concentran procesos empresariales e inventario. Su cobertura no implica equivalencia de reglas: SRD acota el dominio a Juntas de Acción Comunal, registros por junta, permisos operativos y tesorería informativa. La brecha atendida por el prototipo es esa articulación específica, no la pretensión de superar plataformas comerciales en amplitud.

III. METODOLOGÍA

La ampliación del periodo de observación incorporó, con fecha de cierre 19 de octubre de 2026, los recorridos dirigidos del navegador documentados en REVISION_UX_WEB.md y PENDIENTES-WEB-2026-10-16.md, junto con VALIDACION.md y la matriz de

TRAZABILIDAD.md. La unidad de recuento es el archivo de prueba distinto que aprobó sobre una misma imagen web; cada archivo puede contener distinta cantidad de casos. Se registra su condición de ejecución y no se infiere un porcentaje de aceptación del producto.

Se adoptó una investigación aplicada con alcance descriptivo y estudio de caso tecnológico. La unidad de análisis fue el incremento 0.1 de SRD examinado hasta el 19 de octubre de 2026. Las fuentes primarias fueron la especificación de requisitos, el código, las migraciones, contratos OpenAPI, archivos Compose, pruebas automatizadas y registros de una sesión de recuperación operacional. La revisión de literatura se organizó alrededor de multi-tenancy, seguridad, identidad y aplicaciones comparables.

El trabajo siguió cinco fases. Primero, se definió una línea base de 56 RF, 18 RNF y 20 casos de uso para una plataforma dirigida a JAC. Segundo, se separaron responsabilidades mediante el Gateway y servicios independientes. Tercero, se implementaron flujos locales con pruebas unitarias y de interfaz sobre datos sintéticos. Cuarto, se registraron resultados y límites, distinguiendo evidencia del repositorio y comprobaciones de consola. Quinto, se realizó un diagnóstico correctivo sobre migraciones y auditoría en un entorno Docker de demostración.

Los instrumentos fueron una matriz de trazabilidad, casos de prueba, registros de ejecución PHP/Jest/Playwright, sondas Docker y contadores del patrón outbox. No se introdujeron entrevistas, encuestas ni métricas de satisfacción porque no se dispuso de datos verificables de usuarios finales; su aplicación queda para la siguiente etapa de validación del proyecto.

Para controlar errores de interpretación, cada resultado se clasificó por procedencia: prueba automatizada documentada, verificación manual observada o funcionalidad pendiente. No se realizaron pruebas de carga integral ni se equiparó el número de contenedores saludables a un criterio de aceptación de negocio.

Para relacionar pregunta, objetivos y verificación se utilizó una matriz de consistencia operativa. El objetivo de separación por organización se vinculó con las validaciones de contexto del Gateway y de los dominios; el objetivo de trazabilidad, con los contratos de evento y la tabla outbox; y el objetivo de mantenimiento, con los scripts de migración, verificación y respaldo. Este procedimiento permite que una afirmación remita a un archivo y a una prueba localizable, aunque no sustituye una inspección independiente del proyecto.

La selección de casos se concentró en flujos de riesgo: autenticación de miembros, filtrado de recursos por junta, modificación concurrente de carpetas, operaciones financieras informativas, historial de inventario y publicación de auditoría. Los datos del paquete de exposición fueron sintéticos precisamente para no utilizar nombres, correos o movimientos económicos de vecinos durante las verificaciones. Se conservaron como evidencia los resultados explícitos de las suites y las salidas de consola proporcionadas en la sesión.

La revisión bibliográfica se realizó mediante las expresiones multi-tenancy AND security, multi-tenant SaaS AND architecture, community organization information systems y equivalentes en español. Los trabajos [1]–[6] se seleccionaron por tratar separación de recursos, seguridad y arquitectura; los referentes [7]–[9] por su utilidad normativa o técnica, y [10]–[12] como antecedentes de soluciones existentes. Esta selección es exploratoria; no constituye una revisión sistemática exhaustiva ni una evaluación empírica comparativa de los productos.

IV. DISEÑO E IMPLEMENTACIÓN

La evolución posterior al 9 de octubre incorporó sistematización de estados de interfaz a 320 y 1280 píxeles, reintentos ante errores y pruebas de navegación y permisos por seis roles. En Consultar Personas se corrigió la presentación de una ficha previa durante una nueva consulta o después de un 503. Permanecen como componentes parciales la distribución de Android y Windows, la especialización del servicio Reportes y la topología pública con TLS/TURN.

La interfaz web utiliza React 19, TypeScript y Vite; el servidor de entrada utiliza Laravel, mientras MySQL 8.4 mantiene bases independientes por dominio. El prototipo implementa identidad, configuración, registros, auditoría, archivos, calendario, notificaciones, tesorería, inventario y chat. El servicio especializado de reportes permanece diseñado, no implementado. La aplicación Windows Electron carga la web del servidor y requiere red; el cliente Android nativo y una variante Expo Go cubren parte de los recorridos.

El Gateway resuelve la sesión y suministra un contexto firmado de usuario, organización y permisos. Cada microservicio vuelve a revisar el acceso al recurso, de manera que el ocultamiento de botones en React no sea una frontera de seguridad. En base de datos, la separación por dominio se apoya en migraciones y credenciales propias. Las consultas funcionales filtran la junta activa; ningún cliente debería seleccionar libremente la junta de otra persona mediante un identificador enviado en la petición.

El diseño de auditoría utiliza eventos guardados en una tabla outbox con la operación de negocio. Un planificador intenta enviarlos al servicio de auditoría y la recepción deduplica por identificador, lo que evita que un reintento genere registros repetidos. La publicación eventual no garantiza tiempo real estricto: puede haber eventos diferidos o agotados y, por lo tanto, el sistema muestra contadores y funciones de supervisión.

Archivos utiliza almacenamiento privado, límites de tamaño y ClamAV para examinar ciertas cargas. Tesorería utiliza asientos informativos COP y reversos en lugar de editar historia; Inventario conserva movimientos de bienes. El propósito de esas reglas es poder revisar qué ocurrió y cuándo, sin convertir la interfaz en la única garantía.

V. RESULTADOS

A 19 de octubre se documentaron 33 archivos distintos de navegador aprobados entre 140 disponibles, ejecutados sobre una misma imagen web de Docker. Los otros 107 carecen de resultado final en esa imagen, por lo que no se calcula una tasa de aprobación de la suite. La prueba web-empty-states.spec.mjs comprobó 14 secciones de una junta sintética vacía en 320/1280 px, generó 28 capturas, y se ensayaron lecturas lentas y recuperación tras 503 de Consultar Personas. Los archivos service-status.spec.mjs, person-date-ranges.spec.mjs y person-base-filters.spec.mjs aprobaron individualmente después de actualizar expectativas de prueba obsoletas.

Durante una ejecución extensa Docker Desktop dejó de responder; el reinicio controlado recuperó la web (HTTP 200) y registró 23 servicios saludables en ese momento. Es una observación operativa distinta de las pruebas de negocio. Las cargas protegidas por ClamAV quedaron bloqueadas con 503 por firmas vencidas y NO se consideraron verificadas.

En el código del corte analizado se encontraron diez dominios y el Gateway, además de los clientes web, escritorio y móvil. La línea base de trazabilidad contempla 56 RF, 18 RNF y 20 CU con aceptación integral pendiente; las categorías de avance no constituyen una tasa de cobertura. Entre las verificaciones documentadas, dos suites de Archivos aprobaron 32 pruebas con 778 aserciones; un recorrido Chromium de carpetas en español e inglés terminó con 1 caso aprobado, y otro recorrido verificó jerarquía y conflictos. Para Android se registraron 37 pruebas Jest satisfactorias y una compilación debug x86_64.

En la sesión operativa del 9 de octubre, la base de Auditoría carecía inicialmente de la tabla de eventos, de modo que emisores agotaban reintentos. Después de aplicar la migración y respaldar bases se recuperaron 15 eventos de Identidad, ocho de Registros, cuatro de Calendario, tres de Tesorería y tres de Inventario: 33 eventos que terminaron publicados con cero agotados en esos emisores. Las bases de Archivos y Chat recibieron sus migraciones y sus dos planificadores pasaron al estado healthy.

La seguridad de archivos presentó otro problema observable: el actualizador FreshClam fallaba al interpretar un script con saltos CRLF. Tras convertirlo a LF y recrear el actualizador, el motor ClamAV permaneció healthy, se aprobó la prueba de base de firmas y daily.cld alcanzó la versión 28148. Este resultado prueba la actualización de firmas y la accesibilidad del marcador desde Archivos; todavía no equivale a una prueba de aceptación/rechazo de archivos maliciosos de extremo a extremo.

La demostración preparada usa ocho personas pendientes, cuatro eventos, tres asientos y tres bienes, todos ficticios. Una prueba anterior en teléfono Android físico verificó inicio de sesión, MFA, Inicio, Personas, Agenda, Avisos, Perfil y persistencia de sesión mediante Expo Go. No se reportó una medición completa de experiencia de usuario ni una auditoría de producción.

En el frente de autorización, el material técnico indica que el Gateway resuelve la sesión y firma el contexto que llega a cada dominio. Como consecuencia verificable, una interfaz que oculta un botón no concede ni revoca realmente una operación: corresponde a las pruebas de backend comprobar cada permiso. El conjunto revisado tiene una matriz de roles SA, AD, RE, TE, AU y CO y contempla lectura delegada de carpetas; esta regla exige repetir los casos cuando cambia una membresía o se suspende una junta.

En persistencia, el repositorio contiene contratos OpenAPI por dominio y migraciones independientes. La incidencia de Archivos y Chat demostró por qué ambas piezas deben verificarse juntas: la aplicación podía responder al chequeo HTTP mientras su base seguía sin tablas. Tras migrar, los comandos de outbox dejaron de informar ausencia de tabla y ambos planificadores alcanzaron una señal saludable. La comprobación es acotada, pero ofrece una evidencia reproducible de que un indicador superficial no basta para evaluar el comportamiento real.

La interpretación de estos resultados requiere separar tres niveles: correcto despliegue de la infraestructura, ejecución satisfactoria de pruebas de componentes y aceptación del proceso administrativo por la JAC. El primero se midió mediante Docker y migraciones, el segundo mediante suites y recorridos, y el tercero sigue abierto. Esta separación facilita defender los logros disponibles sin asignar porcentajes arbitrarios a los 56 requisitos ni extrapolar a uso público conclusiones obtenidas en una máquina local.

VI. DISCUSIÓN Y LIMITACIONES

La interpretación está limitada por datos sintéticos locales, una muestra dirigida de archivos de navegador y la falta de una evaluación formal con usuarios finales. Los tres criterios de revisión visual integral, operaciones de escritura por rol y regresión de 140 archivos siguen abiertos; adjuntos requieren firmas antivirus vigentes. No se midieron usuarios concurrentes, restauración completa en producción ni funcionamiento WebRTC entre redes con TURN/TLS. Un estado healthy representa disponibilidad observada, no satisfacción de requisitos.

Los resultados evidencian que la estructura modular permite aislar incidencias: el fallo de la tabla de Auditoría detuvo la entrega de eventos, pero sus causas se localizaron sin borrar registros de negocio; la conversión de fin de línea

recuperó el actualizador sin reinstalar MySQL. También muestran un costo operativo real del enfoque: cada servicio necesita migraciones, credenciales, salud y pruebas propias. La separación arquitectónica, por sí sola, no elimina el trabajo de integración.

El patrón outbox fortaleció la trazabilidad frente a fallos temporales, aunque conservar eventos agotados obliga a contar con procedimientos de recuperación y supervisión. La disponibilidad del escáner no elimina los riesgos de formatos, cuotas o derechos de acceso. Del mismo modo, el uso de varias juntas sobre un mismo sistema requiere ensayos sistemáticos de acceso cruzado que no pueden deducirse de una sola demostración satisfactoria.

Las principales limitaciones son la ausencia de un piloto con JAC reales, la falta de cierre integral de requisitos, un servicio de reportes especializado aún pendiente y funciones incompletas en Chat, Android y Windows. No se midió capacidad para cincuenta usuarios concurrentes; tampoco se acreditaron HTTPS de producción, continuidad operativa ni publicación de instaladores firmados. Por ello las cifras de esta investigación describen un incremento y no un producto final aceptado.

VII. CONCLUSIONES Y TRABAJO FUTURO

Para cerrar la evidencia se requiere ejecutar los 140 archivos en la misma imagen final y archivar resultados por archivo/caso, actualizar firmas autorizadamente y repetir cargas seguras, completar combinaciones de autorización y accesibilidad, probar restauración y rendimiento, y realizar evaluación con usuarios mediante autorización y consentimiento. Ninguno de estos trabajos futuros se presenta como ejecutado en el estudio.

El incremento de SRD demuestra una base técnica verificable para gestionar varias organizaciones comunales en una plataforma compartida con controles de contexto, datos por dominio y seguimiento de operaciones. La combinación de pruebas automatizadas, evidencias de Docker y correcciones dirigidas permitió verificar componentes concretos y documentar sus limitaciones.

La siguiente etapa debe completar los requisitos pendientes, ejecutar pruebas cruzadas de permisos entre juntas y de archivos limpios/EICAR, realizar pruebas de carga, aplicar instrumentos a usuarios reales y construir una instalación segura para producción. La principal conclusión metodológica es que implementar una función y demostrar que cumple un criterio son tareas distintas: los resultados deben quedar vinculados a casos, datos, fechas y medios de verificación.

REFERENCIAS

[1] W.-T. Tsai, Q. Shao, Y. Huang y X. Bai, “Towards a scalable and robust multi-tenancy SaaS,” 2010. doi:10.1145/2020723.2020731.

[2] R. Krebs, C. Momm y S. Kounev, “Architectural Concerns in Multi-Tenant SaaS Applications,” 2012. https://www.scitepress.org/PublishedPapers/2012/39576/pdf/index.html

[3] Z. Pervez, S. Lee y Y.-K. Lee, “Multi-Tenant, Secure, Load Disseminated SaaS Architecture,” ICACT, 2010. https://www.icact.org/upload/2010/0265/20100265_finalpaper.pdf

[4] A. Singh, S. Kumar J. y A. Muralidhar, “A Secure Multi-Tenant Model for SaaS System,” International Journal of Computer Applications, vol. 90, núm. 15, 2014. doi:10.5120/15793-4468.

[5] R. Krebs y M. Loesch, “Comparison of Request Admission Based Performance Isolation Approaches in Multi-tenant SaaS Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/49488/pdf/index.html

[6] M. Reinhardt, S. T. Ruehl, S. A. W. Verclas, U. Andelfinger y A. Schütte, “Architectural Design of a Deployment Platform to Provision Mixed-tenancy SaaS-Applications,” 2014. https://www.scitepress.org/PublishedPapers/2014/48138/pdf/index.html

[7] Congreso de Colombia, “Ley 2166 de 2021: organismos de acción comunal,” 2021. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=184758

[8] OWASP Foundation, “Application Security Verification Standard (ASVS),” versión 5.0.0, 2025. https://github.com/OWASP/ASVS

[9] NIST, “Digital Identity Guidelines: Authentication and Authenticator Management,” SP 800-63B-4, 2025. doi:10.6028/NIST.SP.800-63B-4.

[10] CiviCRM, “CiviCRM Core: constituent relationship management for non-profit organizations,” https://github.com/civicrm/civicrm-core

[11] Odoo, “Inventory: Features,” https://www.odoo.com/app/inventory-features

[12] Frappe Technologies, “ERPNext,” https://erpnext.com

[13] J. S. González Horta, “SRD: README, VALIDACION, TRAZABILIDAD, ARQUITECTURA y código fuente,” repositorio del proyecto, revisión de trabajo hasta el 19 de octubre de 2026.

DECLARACIÓN DE HERRAMIENTAS

La redacción y la revisión lingüística contaron con asistencia digital. El autor contrastó el contenido técnico con el código, los registros de prueba y las fuentes citadas; no se generaron datos de encuestas ni resultados experimentales inexistentes.